Rapport Itération numéro 2

Identification des membres de l'équipe

Membre 1

- Ardy, Yahya
- yahya.ardy.1@ens.etsmtl.ca
- AT73950
- xMrYahya

Membre 2

- Boulianne, Alex
- alex.boulianne.1@ens.etsmtl.ca
- AT72810
- c4tiki

Membre 3

- Gamache, Alexandre
- alexandre.gamache.1@ens.etsmtl.ca
- AU74150
- AlexandreG17

Membre 4

- Hoffmann, Raphaël
- raphael.hoffmann.1@ens.etsmtl.ca
- AU65470
- WishPib

Membre 5

- Kandil, Kassem
- kassem.kandil.1@ens.etsmtl.ca
- AU84220
- kassem0303,kassem03-ets

Exigences

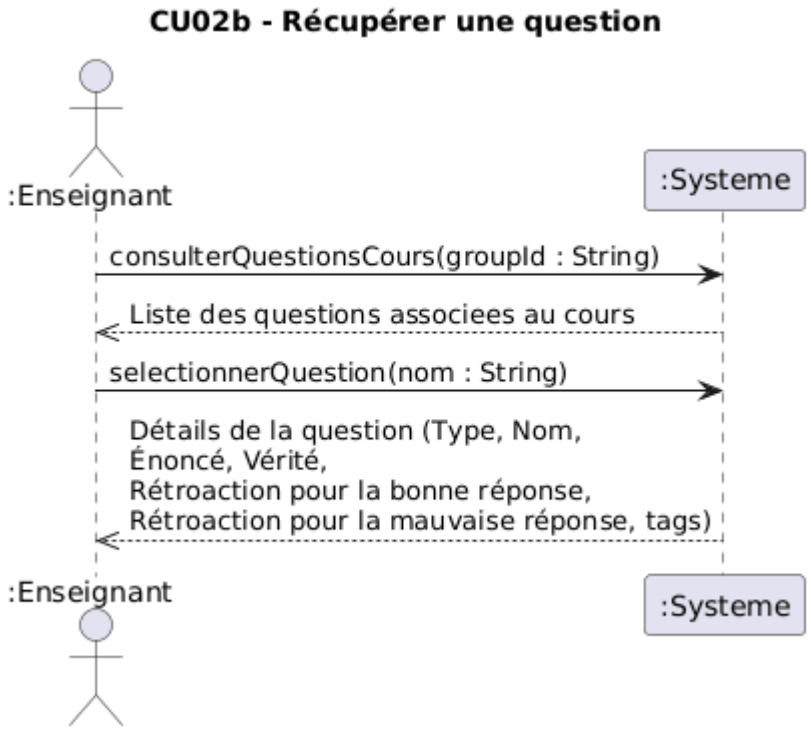
Exigence	Responsable
Ajout des DSS et contrats d'operation des nouveaux CU	Alexandre Gamache
Mise a jour du MDD	Alexandre Gamache
CU02b,c code, tests et RDCUs	Yahya Ardy, Alex Boulianne

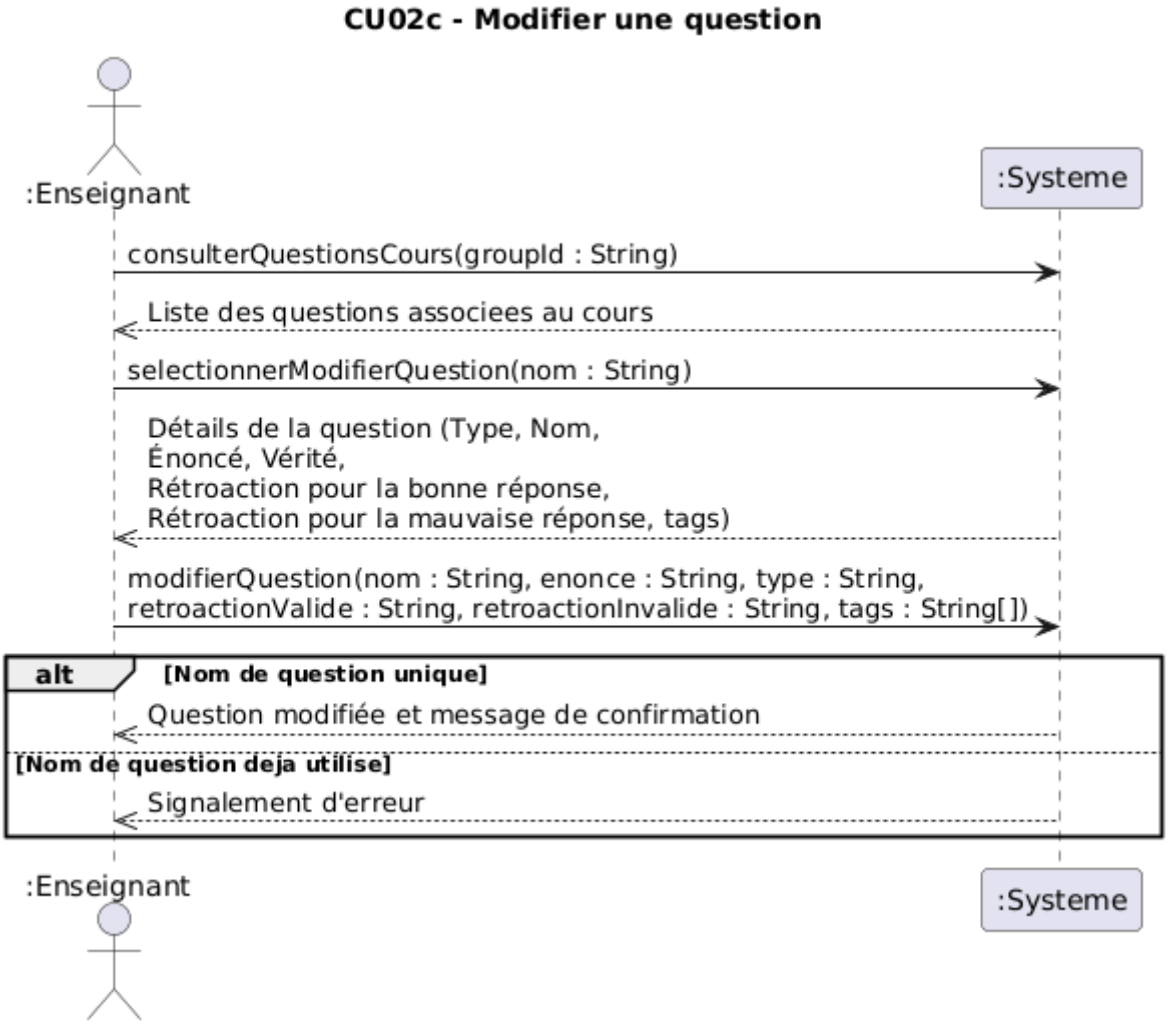
Exigence	Responsable
CU05a,b,c,d code, tests et RDCUs	Kassem Kandil, raphael hoffmann

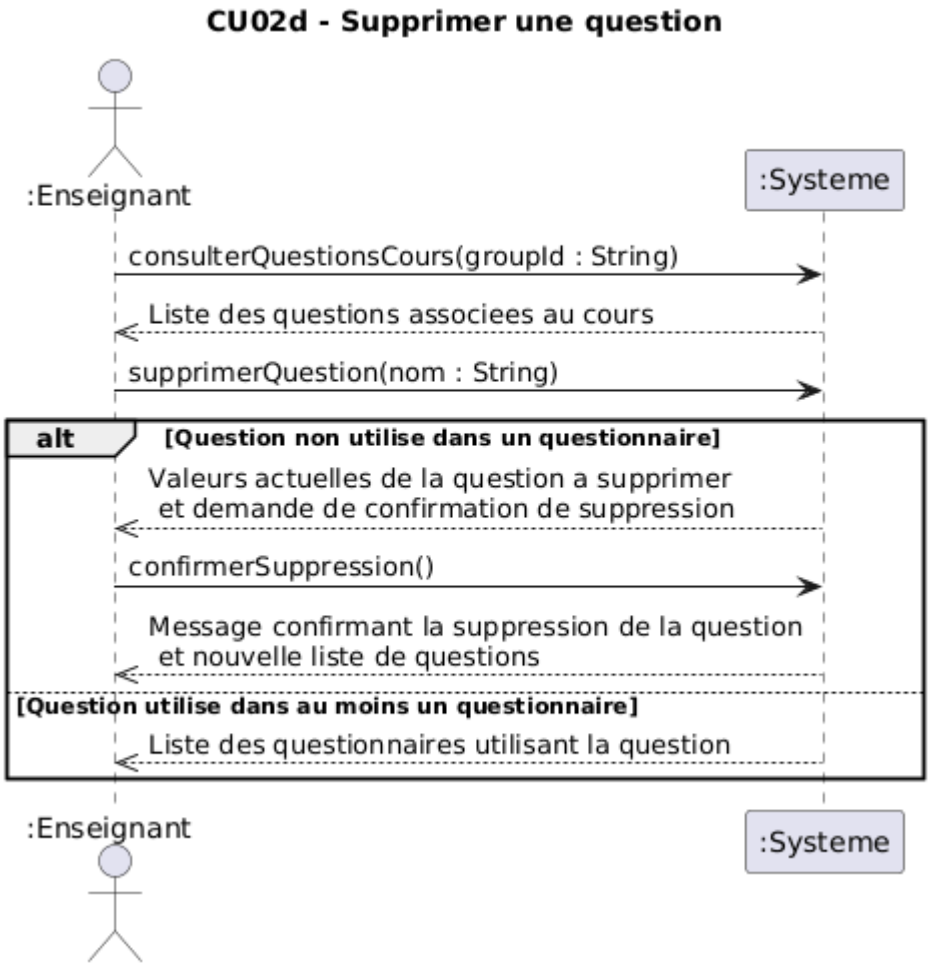
Modèle du domaine (MDD)

> Le MDD est cumulatif : vous devez y ajouter des éléments à chaque itération (ou corriger les erreurs), selon la portée (et votre meilleure compréhension du problème) visée par votre solution. Utilisez une légende dans le MDD pour indiquer la couleur de chaque itération afin de faire ressortir les changements (ce n'est pas toujours possible pour les associations et les attributs). Voir les stéréotypes personnalisés : https://plantuml.com/fr/class-diagram et comment faire une légende avec couleurs en PlantUML.

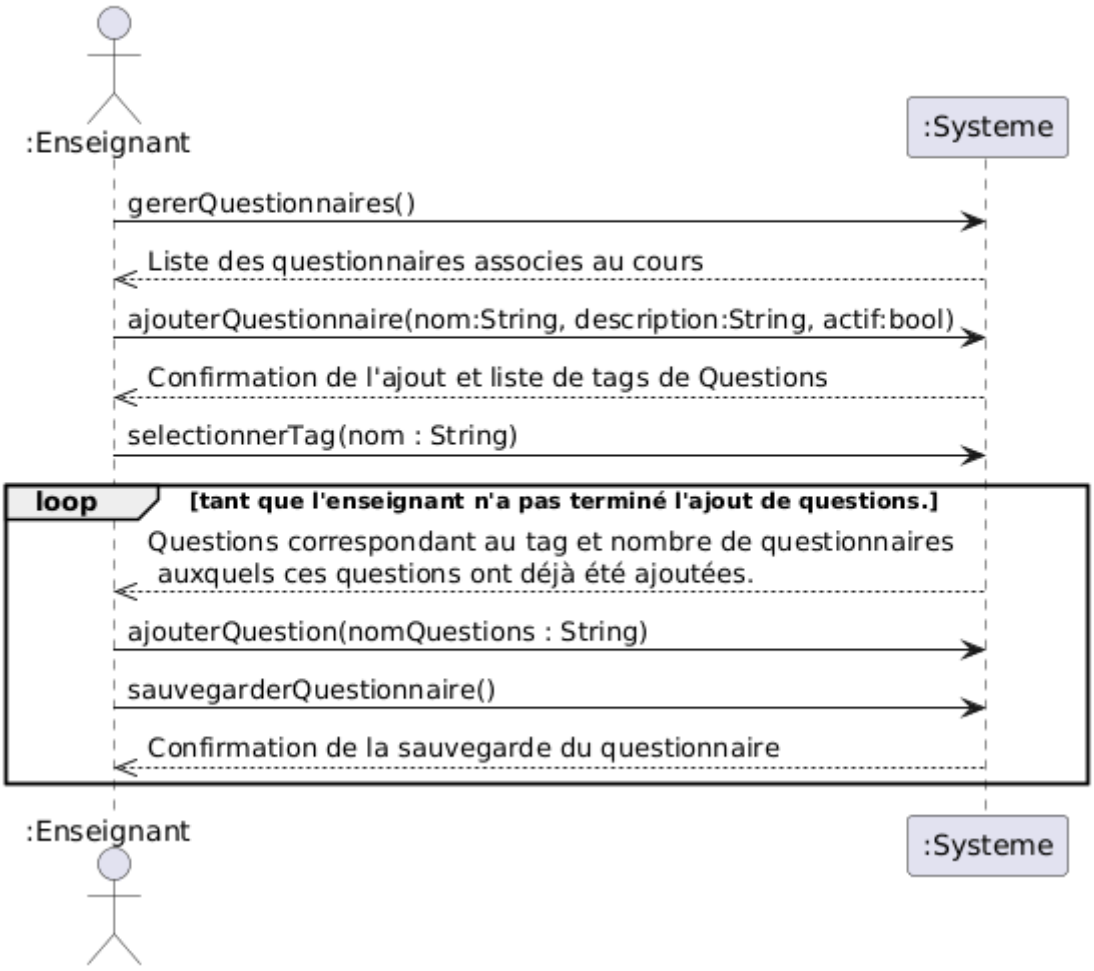
Diagramme de séquence système (DSS)



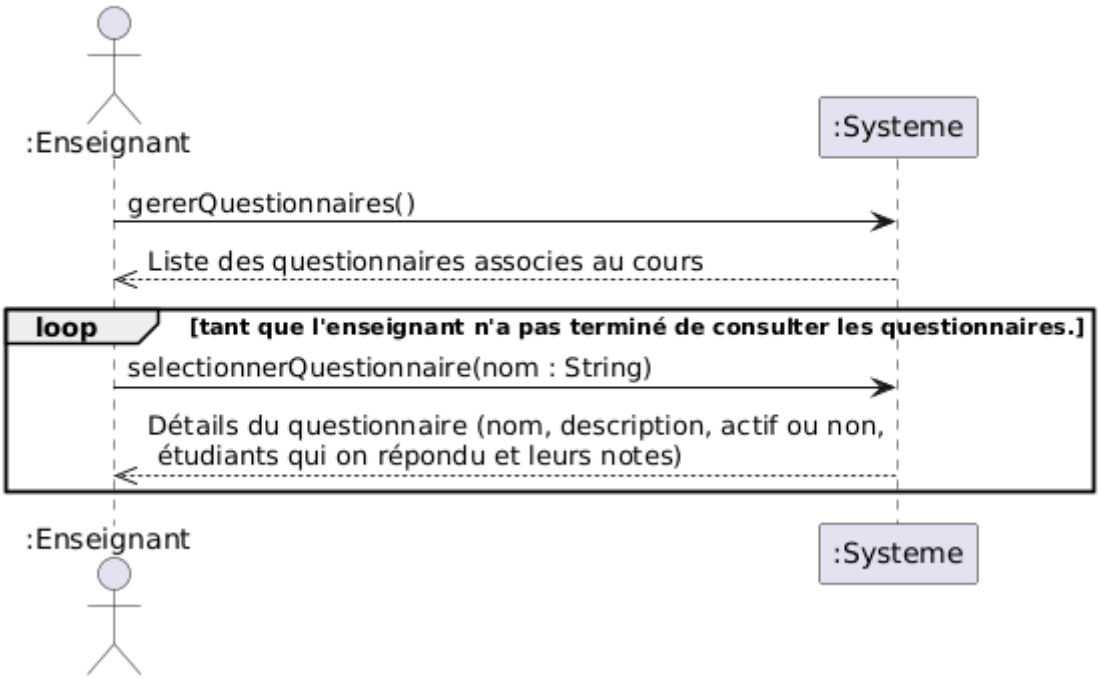

CU02c - Modifier une question
:Enseignant
:Systeme
consulterQuestionsCours(groupId : String)
Liste des questions associees au cours
selectionnerModifierQuestion(nom : String)
Détails de la question (Type, Nom,
Énoncé, Vérité,
Rétroaction pour la bonne réponse,
Rétroaction pour la mauvaise réponse, tags)
modifierQuestion(nom : String, enonce : String, type : String,
retroactionValide : String, retroactionInvalide : String, tags : String[])
alt
[Nom de question unique]
Question modifiée et message de confirmation
[Nom de question deja utilise]
Signalement d'erreur
:Enseignant
:Systeme

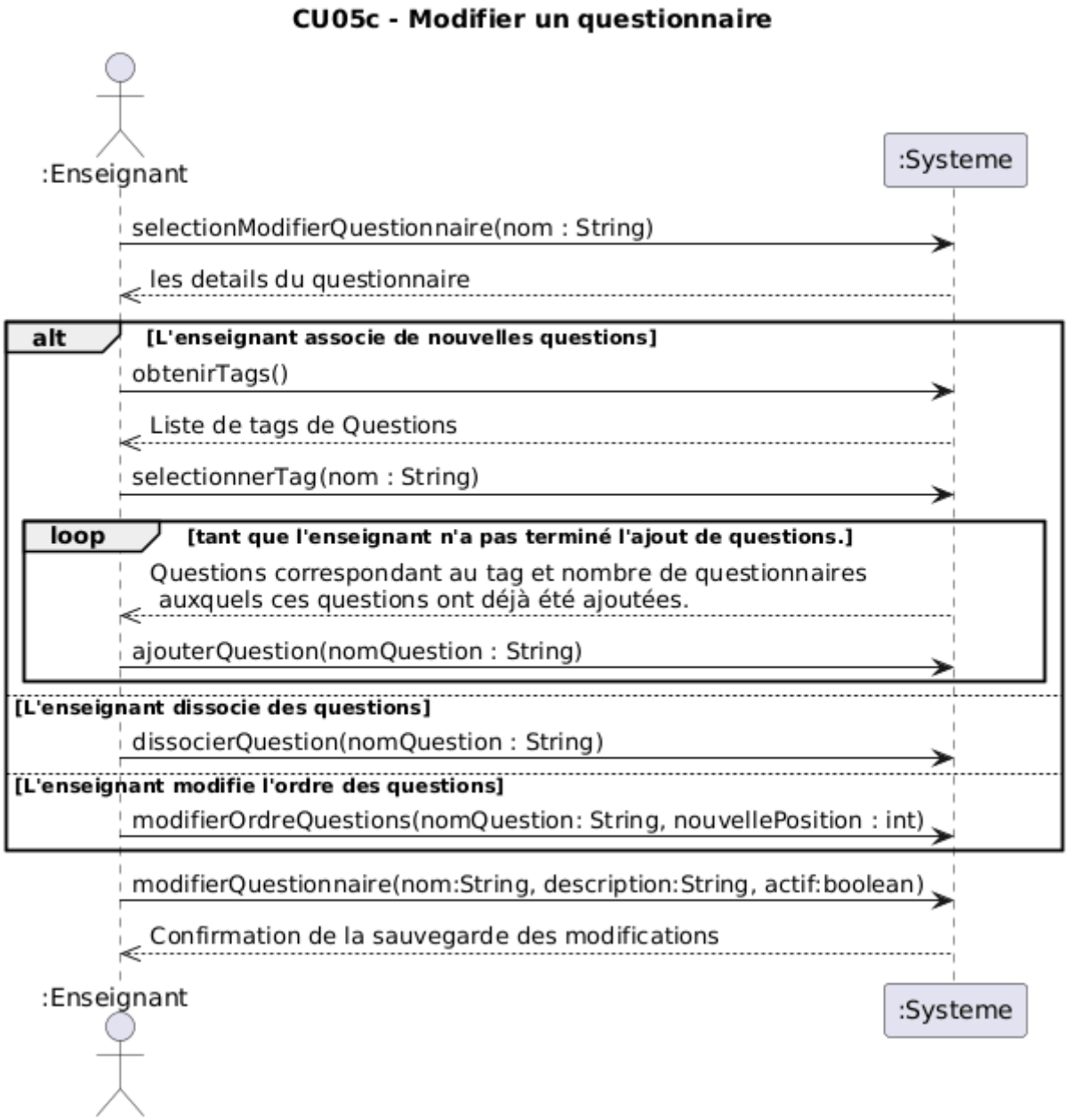

CU02d - Supprimer une question
:Enseignant
:Systeme
consulterQuestionsCours(groupId : String)
Liste des questions associees au cours
supprimerQuestion(nom : String)
alt
[Question non utilise dans un questionnaire]
Valeurs actuelles de la question a supprimer
et demande de confirmation de suppression
confirmerSuppression()
Message confirmant la suppression de la question
et nouvelle liste de questions
[Question utilise dans au moins un questionnaire]
Liste des questionnaires utilisant la question
:Enseignant
:Systeme

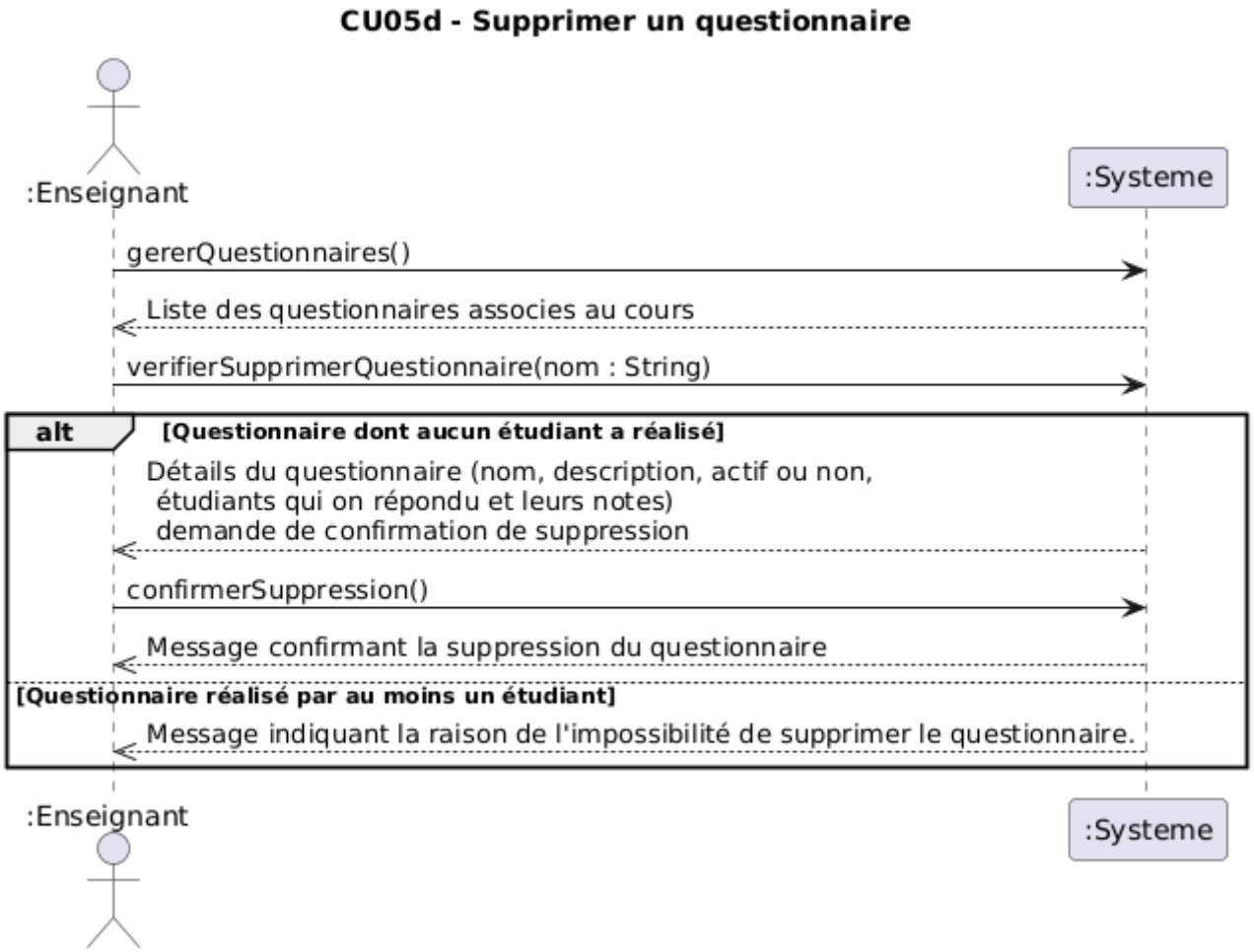
CU05a - Ajouter un questionnaire



CU05b - Afficher un questionnaire




CU05c - Modifier un questionnaire
:Enseignant
:Systeme
selectionModifierQuestionnaire(nom : String)
les details du questionnaire
alt
[L'enseignant associe de nouvelles questions]
obtenirTags()
Liste de tags de Questions
selectionnerTag(nom : String)
loop
[tant que l'enseignant n'a pas terminé l'ajout de questions.]
Questions correspondant au tag et nombre de questionnaires auxquels ces questions ont déjà été ajoutées.
ajouterQuestion(nomQuestion : String)
[L'enseignant dissocie des questions]
dissocierQuestion(nomQuestion : String)
[L'enseignant modifie l'ordre des questions]
modifierOrdreQuestions(nomQuestion: String, nouvellePosition : int)
modifierQuestionnaire(nom:String, description:String, actif:boolean)
Confirmation de la sauvegarde des modifications
:Enseignant
:Systeme



Contrats

Contrat CO10 - Afficher la liste de question d'un cours

Opération: consulterQuestionsCours(groupId : String)
Références croisées:

- CU02b - Récuperer une question
- MDD - Enseignant, Cours, Question
- DSS - Récuperer Question **Préconditions:**
- Un cours est sélectionné

PostConditions:

- Aucune postcondition

Contrat CO11 - Afficher une question de la liste de question d'un cours

---

Opération: selectionnerQuestion(nom : String)
Références croisées:

- CU02b - Récuperer une question
- MDD - Enseignant, Cours, Question

- DSS - Récuperer Question **Préconditions:**
- La liste de question d'un cours est récupéré

PostConditions:

- Aucune postcondition

Contrat CO12 - Demande de suppression de question

Opération: supprimerQuestion(groupid : String, nom : String)
Références croisées:

- CU02d - Supprimer une question
- MDD - Enseignant, Cours, Question
- DSS - Supprimer question **Préconditions:**
- Une liste de cours est selectionné

PostConditions:

- Aucune post condition

Contrat CO13 - Confirmation de suppression de question

Opération: confirmerSuppression()
Références croisées:

- CU02d - Supprimer une question
- MDD - Enseignant, Cours, Question
- DSS - Supprimer question **Préconditions:**
- Une demande de suppression est débuté pour un cours

PostConditions:

- La question selectionné a été dissocié du QuestionStore de la question selectionné
- La question selectionné a été supprimée

Contrat CO14 - Modification de question

Opération: modifierQuestion(nom, enonce, type,retroactionValide, retroactionInvalide, tags)
Références croisées:

- CU02c - Modifier une question
- MDD - Enseignant, Cours, Question
- DSS - Modifier une question **Préconditions:**
- Le nouveau nom de la question a modifier est valide
- Une question est sélectionné

PostConditions:

- nom a été associé dans Question.nom de la question sélectionné
- enonce a été associé dans Question.enonce de la question sélectionné
- type a été associé dans Question.type de la question sélectionné
- retroactionValide a été associé dans Question.retroactionValide de la question sélectionné
- retroactionInvalide a été associé dans Question.retroactionInvalide de la question sélectionné

Contrat CO15 - Gerer les questionnaires

Opération: gererQuestionnaires()
Références croisées:

- CU05a - Ajouter un questionnaire
- MDD - Enseignant, Cours, Question
- DSS -Ajouter un questionnaire **Préconditions:**
- Un cours est sélectionné

PostConditions:

- Aucune postCondition (les questionnaires sont seulement retourné par le controlleur)

Contrat CO16 - Ajouter un questionnaire

Opération: ajouterQuestionnaire(nom:String, description:String, actif:boolean)
Références croisées:

- CU05a - Ajouter un questionnaire
- MDD - Enseignant, Cours, Question
- DSS -Ajouter un questionnaire **Préconditions:**
- Un cours est sélectionné
- L'option ajouter Questionnaire est selectionné

PostConditions:

- une instance q de questionnaire a été créé
- q a été associé au cours sélectionné
- nom a été assigné a q.nom
- description a été associé a q.description
- actif a été associé a q.actif

Contrat CO17 - Selectionner un tag

Opération: selectionnerTag(nomTag:String)
Références croisées:

- CU05a - Ajouter un questionnaire
- MDD - Enseignant, Cours, Question
- DSS -Ajouter un questionnaire **Préconditions:**
- Un tag est selectionné

PostConditions:

- Une instance questionnaireTemp de Questionnaire a été crée

Contrat CO18 - Ajouter une question

Opération: ajouterQuestion(nomQuestion:String)
Références croisées:

- CU05a - Ajouter un questionnaire
- MDD - Enseignant, Cours, Question
- DSS -Ajouter un questionnaire **Préconditions:**
- Un questionnaire est sélectionné

PostConditions:

- une question a été ajouté au questionnaireTemp avec la correspondance du nomQuestion

Contrat CO19 - Sauvegarder un questionnaire

Opération: sauvegarderQuestionnaire()
Références croisées:

- CU05a - Ajouter un questionnaire
- MDD - Enseignant, Cours, Question
- DSS -Ajouter un questionnaire **Préconditions:**
- Un questionnaire temporaire est selectionné

PostConditions:

- une question a été ajouté au questionnaireTemp avec la correspondance du nomQuestion

Contrat CO20 - Gérer les questionnaires

Opération: gererQuestionnaires() Références croisées:

- CU05b - afficher un questionnaire
- DSS - Afficher un questionnaire **Préconditions:**
- L'enseignant est authentifié
- Un cours est sélectionné

Postconditions:

- Aucune postcondition (La liste des questionnaire est affiché)

Contrat CO21 - Sélectionner un questionnaire

Opération: selectionnerQuestionnaire(nom : String) Références croisées:

- CU05b - Afficher un questionnaire
- DSS - Afficher un questionnaire
- CO20 - Gérer les questionnaires

Préconditions:

- Aucune précondition

PostConditions:

- Aucune postcondition (les détails du questionnaire sélectionné sont affiché)

Contrat CO22 - Sélectionner un questionnaire a modifier

Opération: selectionModifierQuestionnaire(nom : String) Références croisées:

- CU05c - Modifier un questionnaire
- DSS - Modifier un questionnaire
- CO20 - Gérer les questionnaires

Préconditions:

- L'enseignant est authentifié
- L'enseignant appuie sur gérer les questionnaires

Postconditions:

- f

Contrat CO23 - Obtenir les tags

Opération: obtenirTags() Références croisées:

- CU05c - Modifier un questionnaire
- DSS - Modifier un questionnaire
- CO22 Sélectionner un questionnaire a modifier

Préconditions:

- Un questionnaire est sélectionné avec selectionModifierQuestionnaire(nom : String)

Postconditions:

- Aucun postcondition (la liste des tags a été affiché)

Contrat CO24 - dissocier une question

Opération: dissocierQuestion(nomQuestion : String) Références croisées:

- CU05c - Modifier un questionnaire
- DSS - Modifier un questionnaire

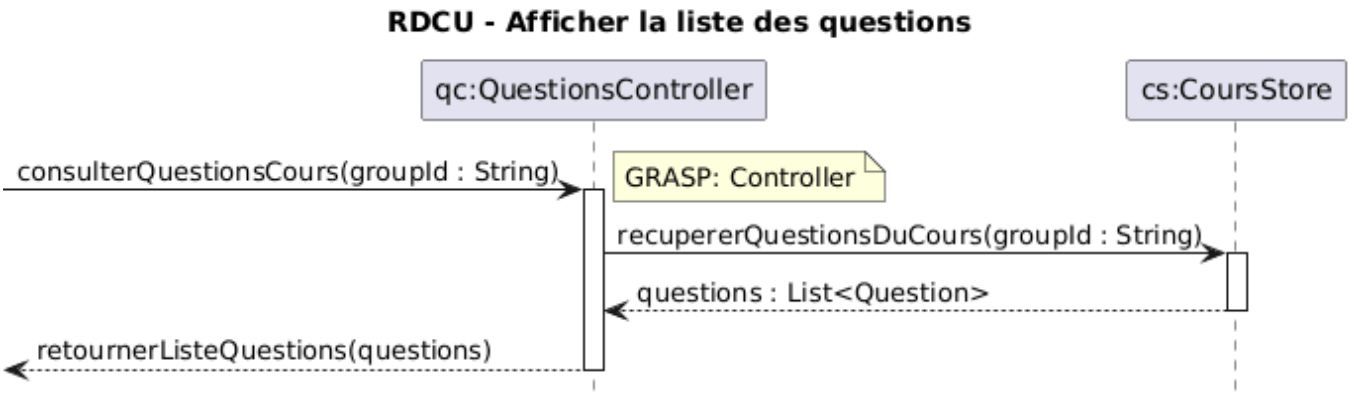
Préconditions:

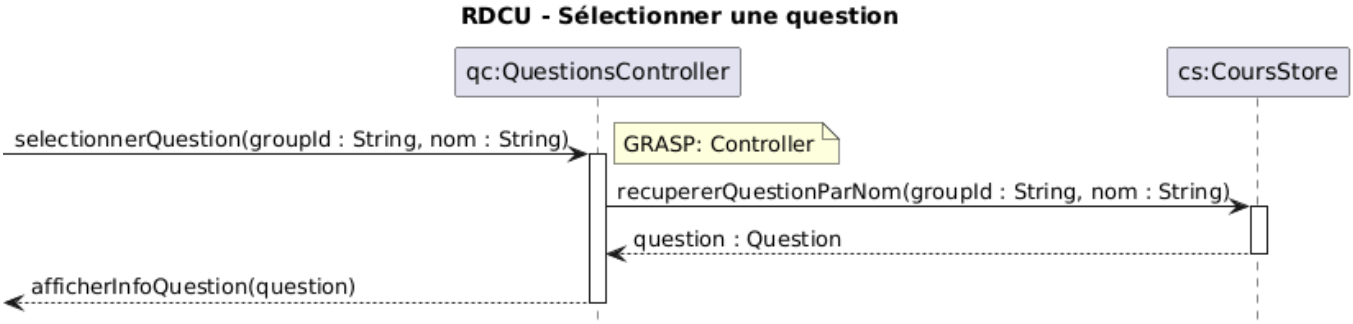
- Un questionnaire est sélectionné avec selectionModifierQuestionnaire(nom : String)

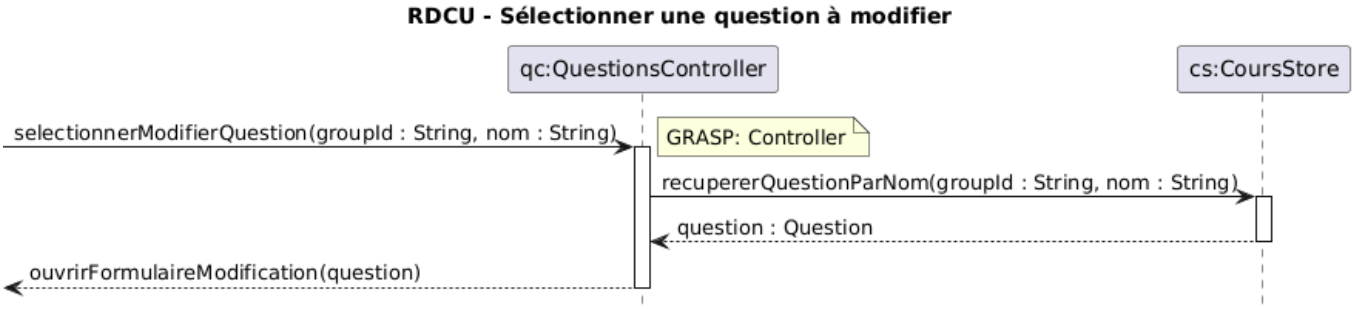
Postconditions:

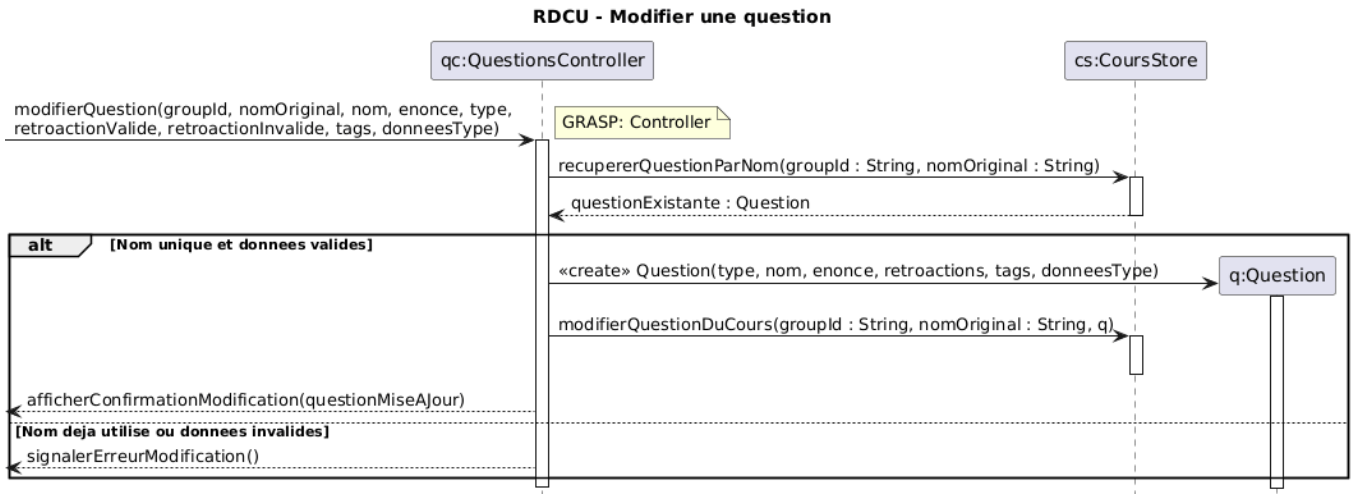
Réalisation de cas d'utilisation (RDCU)

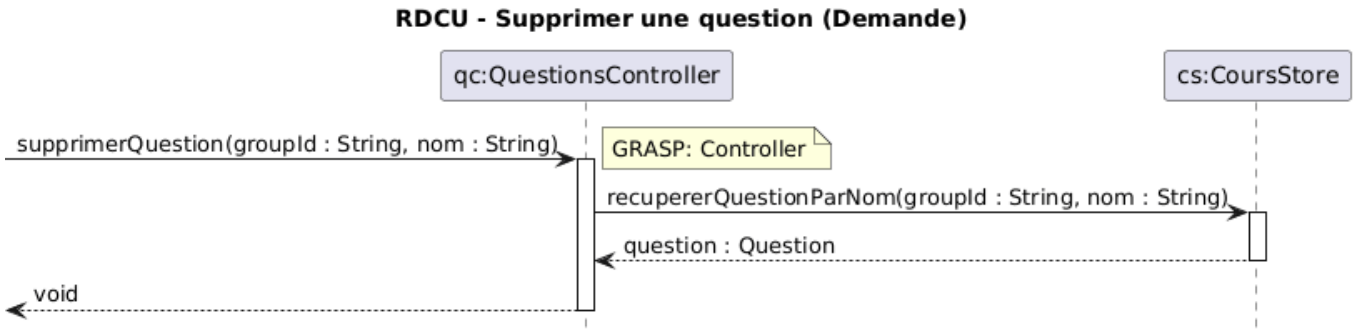
RDCU

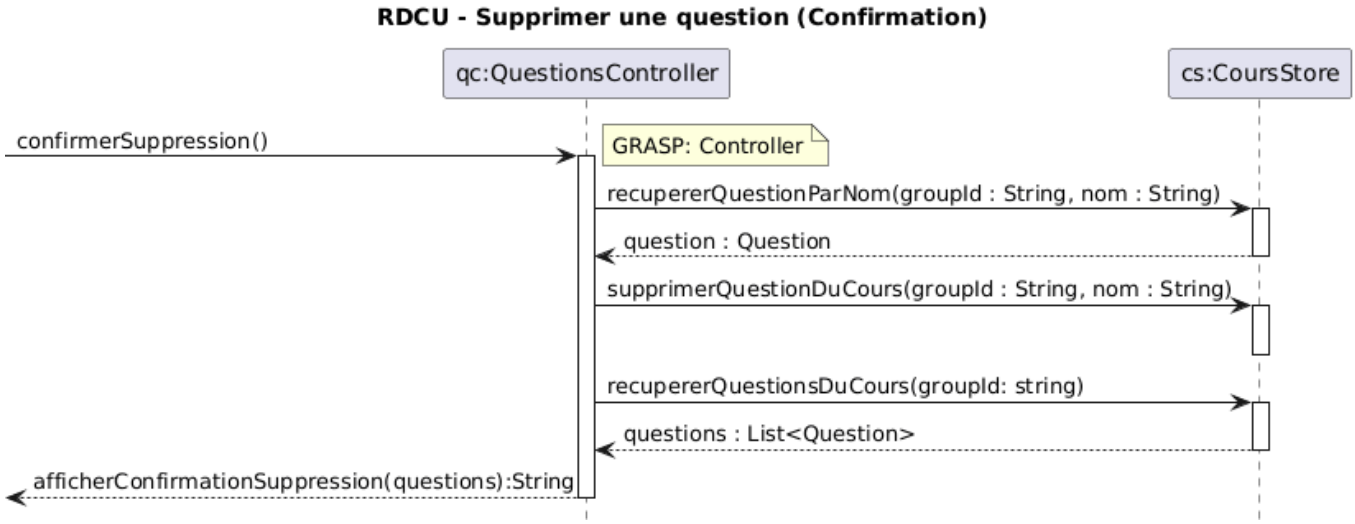


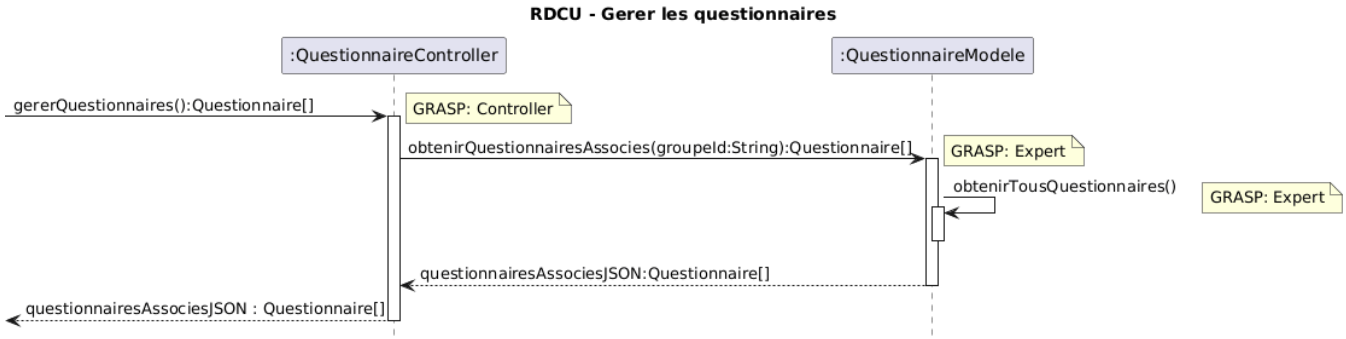


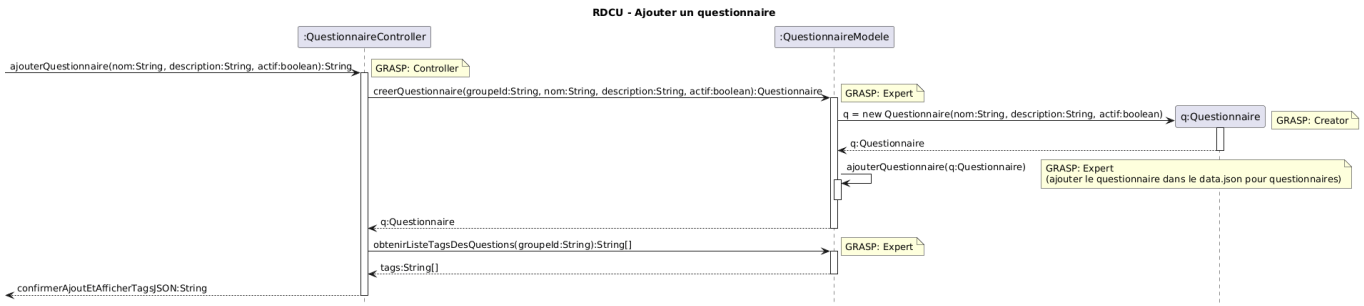


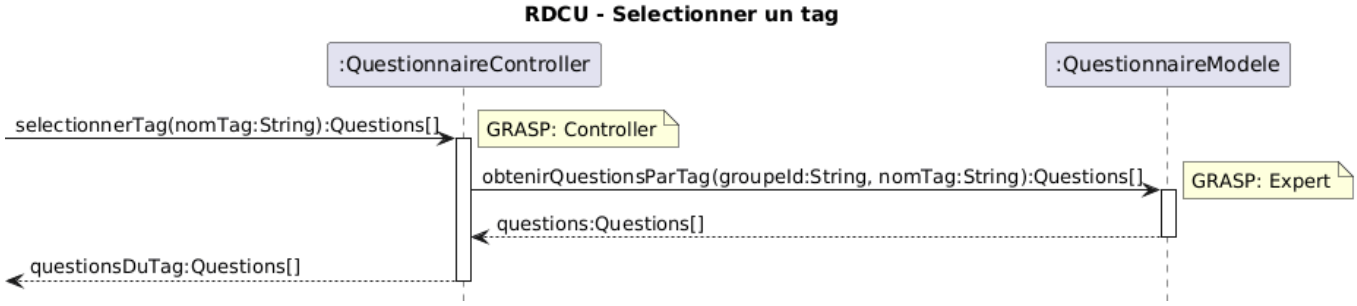

RDCU - Modifier une question
qc:QuestionsController
cs:CoursStore
modifierQuestion(groupId, nomOriginal, nom, enonce, type, retroactionValide, retroactionInvalide, tags, donneesType)
GRASP: Controller
recupererQuestionParNom(groupId : String, nomOriginal : String)
questionExistante : Question
alt
[Nom unique et donnees valides]
«create» Question(type, nom, enonce, retroactions, tags, donneesType)
q:Question
modifierQuestionDuCours(groupId : String, nomOriginal : String, q)
afficherConfirmationModification(questionMiseAJour)
[Nom deja utilise ou donnees invalides]
signalerErreurModification()

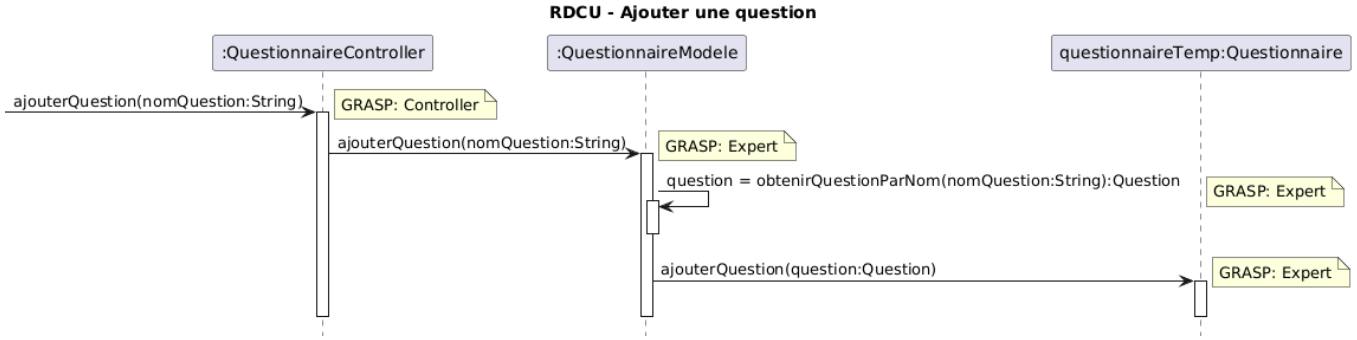

RDCU - Supprimer une question (Demande)
qc:QuestionsController
cs:CoursStore
supprimerQuestion(groupId : String, nom : String)
GRASP: Controller
recupererQuestionParNom(groupId : String, nom : String)
question : Question
void

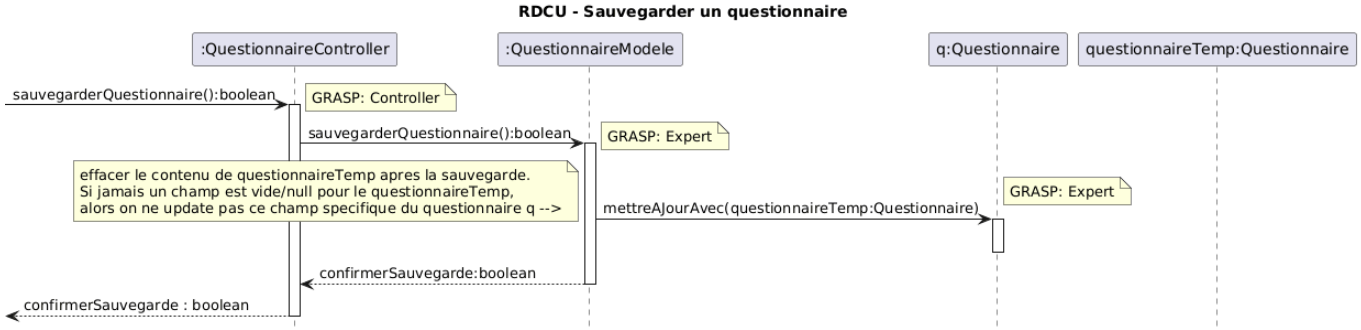

RDCU - Supprimer une question (Confirmation)
qc:QuestionsController
cs:CoursStore
confirmerSuppression()
GRASP: Controller
recupererQuestionParNom(groupId : String, nom : String)
question : Question
supprimerQuestionDuCours(groupId : String, nom : String)
recupererQuestionsDuCours(groupId: string)
questions : List<Question>
afficherConfirmationSuppression(questions):String

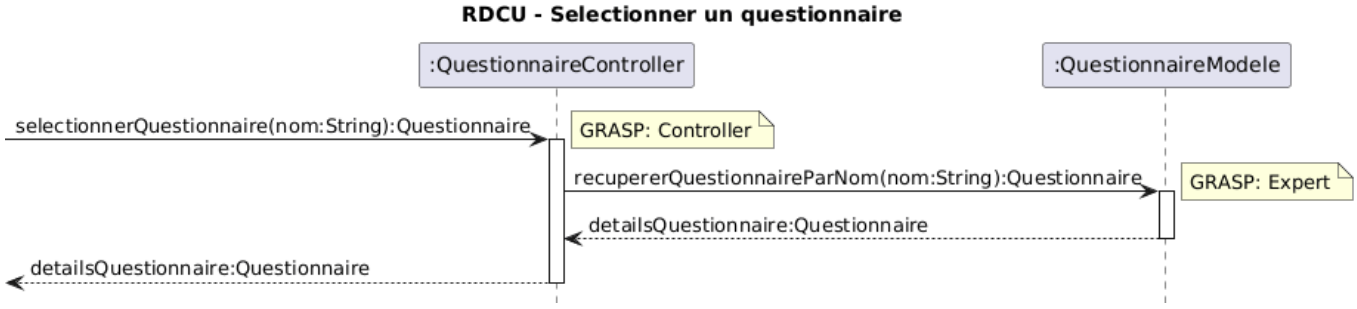

RDCU - Gerer les questionnaires
:QuestionnaireController
:QuestionnaireModele
gererQuestionnaires():Questionnaire[]
GRASP: Controller
obtenirQuestionnairesAssocies(groupeId:String):Questionnaire[]
GRASP: Expert
obtenirTousQuestionnaires()
GRASP: Expert
questionnairesAssociesJSON:Questionnaire[]
questionnairesAssociesJSON : Questionnaire[]

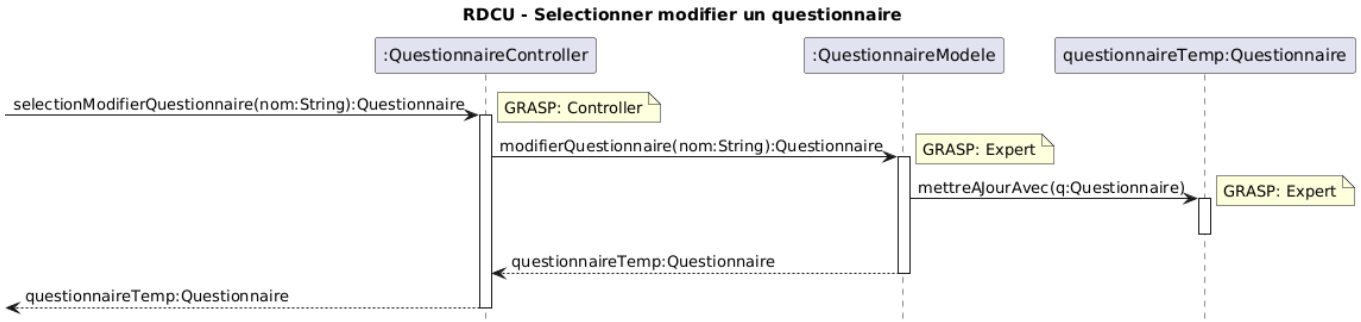

RDCU - Ajouter un questionnaire
:QuestionnaireController
:QuestionnaireModele
ajouterQuestionnaire(nom:String, description:String, actif:boolean):String
GRASP: Controller
creerQuestionnaire(groupeId:String, nom:String, description:String, actif:boolean):Questionnaire
GRASP: Expert
q = new Questionnaire(nom:String, description:String, actif:boolean)
q:Questionnaire
GRASP: Creator
q:Questionnaire
ajouterQuestionnaire(q:Questionnaire)
GRASP: Expert
(ajouter le questionnaire dans le data.json pour questionnaires)
q:Questionnaire
obtenirListeTagsDesQuestions(groupeId:String):String[]
GRASP: Expert
tags:String[]
confirmerAjoutEtAfficherTagsJSON:String

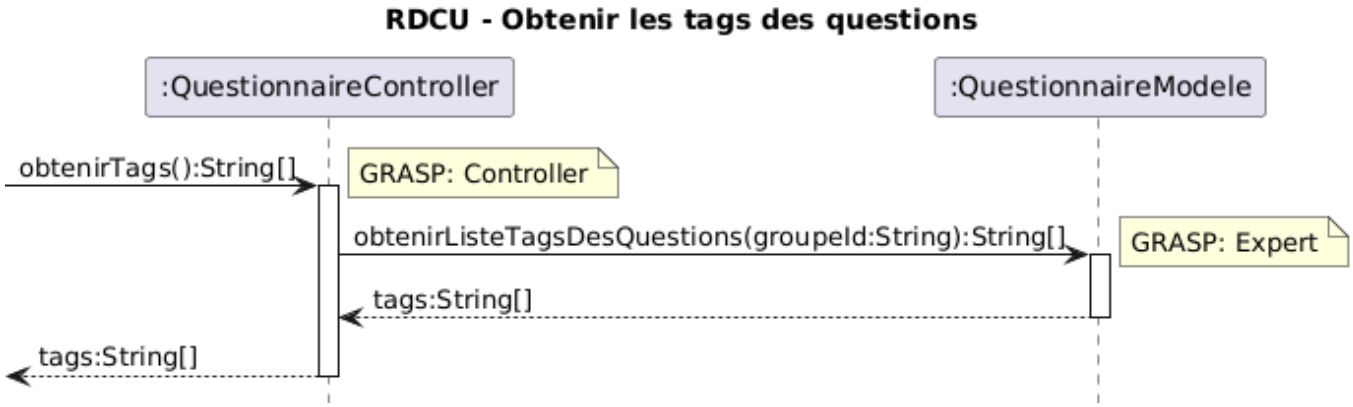

RDCU - Selectionner un tag
:QuestionnaireController
:QuestionnaireModele
selectionnerTag(nomTag:String):Questions[]
GRASP: Controller
obtenirQuestionsParTag(groupeId:String, nomTag:String):Questions[]
GRASP: Expert
questions:Questions[]
questionsDuTag:Questions[]

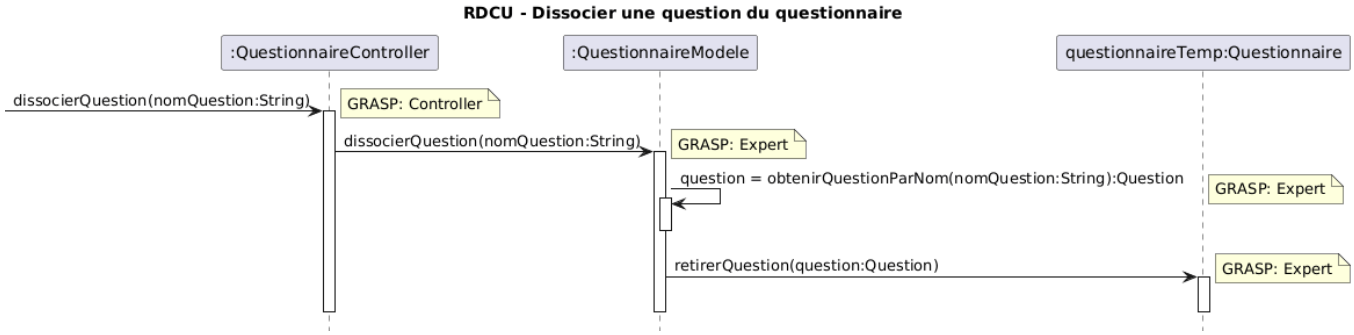

RDCU - Ajouter une question
:QuestionnaireController
:QuestionnaireModele
questionnaireTemp:Questionnaire
ajouterQuestion(nomQuestion:String)
GRASP: Controller
ajouterQuestion(nomQuestion:String)
GRASP: Expert
question = obtenirQuestionParNom(nomQuestion:String):Question
GRASP: Expert
ajouterQuestion(question:Question)
GRASP: Expert

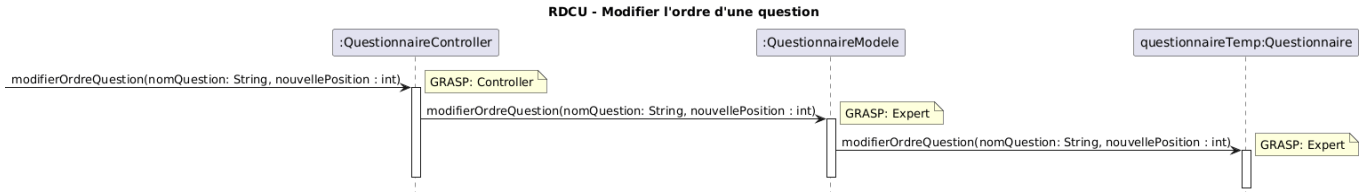

RDCU - Sauvegarder un questionnaire
:QuestionnaireController
:QuestionnaireModele
q:Questionnaire
questionnaireTemp:Questionnaire
sauvegarderQuestionnaire():boolean
GRASP: Controller
sauvegarderQuestionnaire():boolean
GRASP: Expert
effacer le contenu de questionnaireTemp apres la sauvegarde.
Si jamais un champ est vide/null pour le questionnaireTemp,
alors on ne update pas ce champ specifique du questionnaire q -->
GRASP: Expert
mettreAJourAvec(questionnaireTemp:Questionnaire)
confirmerSauvegarde:boolean
confirmerSauvegarde : boolean

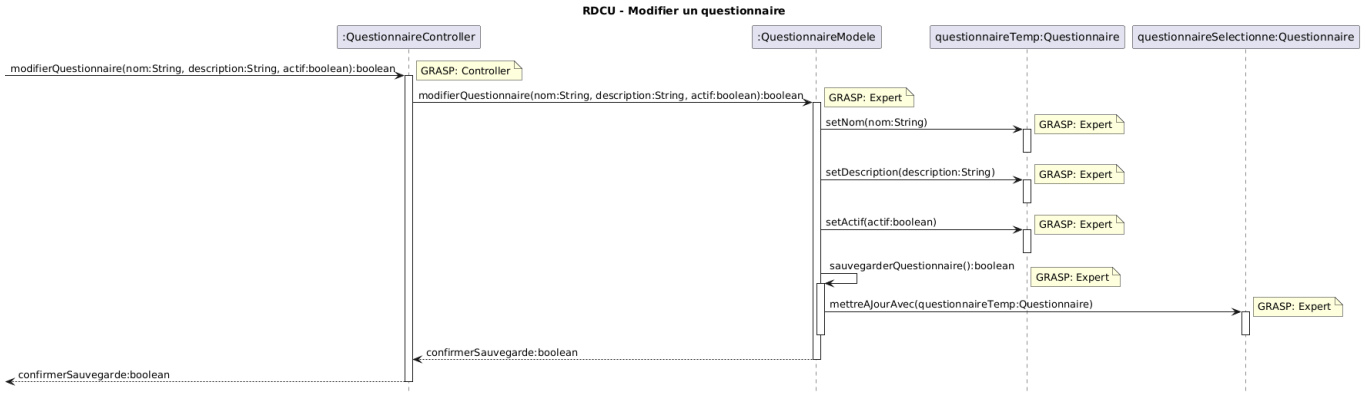

RDCU - Selectionner un questionnaire
:QuestionnaireController
:QuestionnaireModele
selectionnerQuestionnaire(nom:String):Questionnaire
GRASP: Controller
recupererQuestionnaireParNom(nom:String):Questionnaire
GRASP: Expert
detailsQuestionnaire:Questionnaire
detailsQuestionnaire:Questionnaire

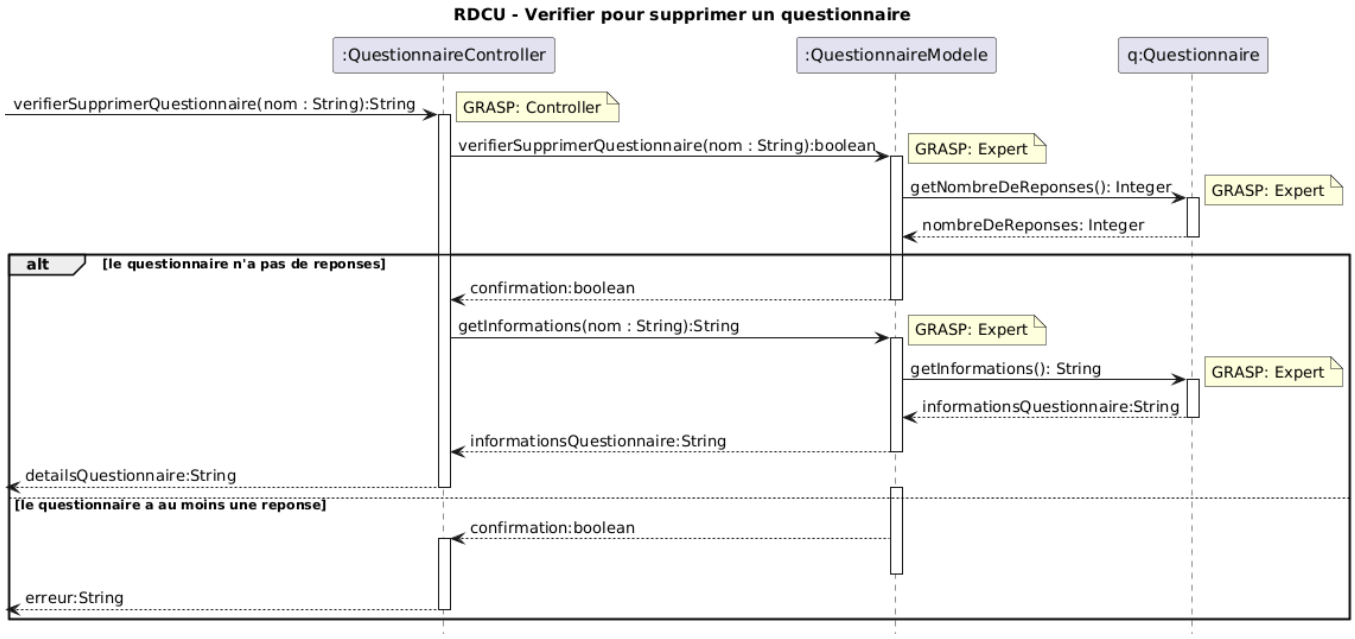

RDCU - Selectionner modifier un questionnaire
:QuestionnaireController
:QuestionnaireModele
questionnaireTemp:Questionnaire
selectionModifierQuestionnaire(nom:String):Questionnaire
GRASP: Controller
modifierQuestionnaire(nom:String):Questionnaire
GRASP: Expert
mettreAJourAvec(q:Questionnaire)
GRASP: Expert
questionnaireTemp:Questionnaire
questionnaireTemp:Questionnaire


RDCU - Obtenir les tags des questions
:QuestionnaireController
:QuestionnaireModele
obtenirTags():String[]
GRASP: Controller
obtenirListeTagsDesQuestions(groupeId:String):String[]
GRASP: Expert
tags:String[]
tags:String[]


RDCU - Dissocier une question du questionnaire
:QuestionnaireController
:QuestionnaireModele
questionnaireTemp:Questionnaire
dissocierQuestion(nomQuestion:String)
GRASP: Controller
dissocierQuestion(nomQuestion:String)
GRASP: Expert
question = obtenirQuestionParNom(nomQuestion:String):Question
GRASP: Expert
retirerQuestion(question:Question)
GRASP: Expert


RDCU - Modifier l'ordre d'une question
:QuestionnaireController
:QuestionnaireModele
questionnaireTemp:Questionnaire
modifierOrdreQuestion(nomQuestion: String, nouvellePosition : int)
GRASP: Controller
modifierOrdreQuestion(nomQuestion: String, nouvellePosition : int)
GRASP: Expert
modifierOrdreQuestion(nomQuestion: String, nouvellePosition : int)
GRASP: Expert


RDCU - Modifier un questionnaire
:QuestionnaireController
:QuestionnaireModele
questionnaireTemp:Questionnaire
questionnaireSelectionne:Questionnaire
modifierQuestionnaire(nom:String, description:String, actif:boolean):boolean
GRASP: Controller
modifierQuestionnaire(nom:String, description:String, actif:boolean):boolean
GRASP: Expert
setNom(nom:String)
GRASP: Expert
setDescription(description:String)
GRASP: Expert
setActif(actif:boolean)
GRASP: Expert
sauvegarderQuestionnaire():boolean
GRASP: Expert
mettreAJourAvec(questionnaireTemp:Questionnaire)
GRASP: Expert
confirmerSauvegarde:boolean
confirmerSauvegarde:boolean



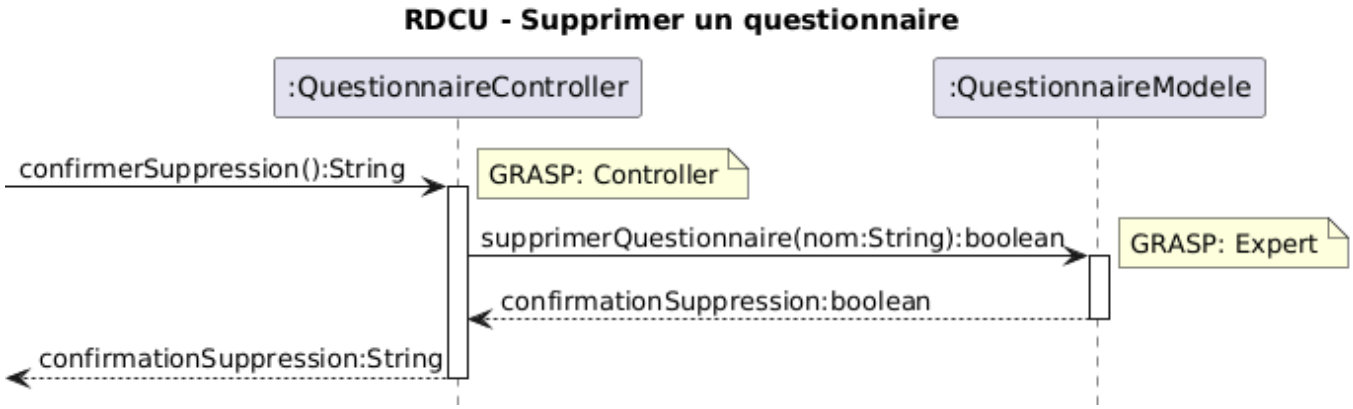


Diagramme de classe logicielle (DCL)

> Facultatif, mais fortement suggéré Ce diagramme vous aidera à planifier l'ordre d'implémentation des classes. Très utile lorsqu'on utilise TDD.

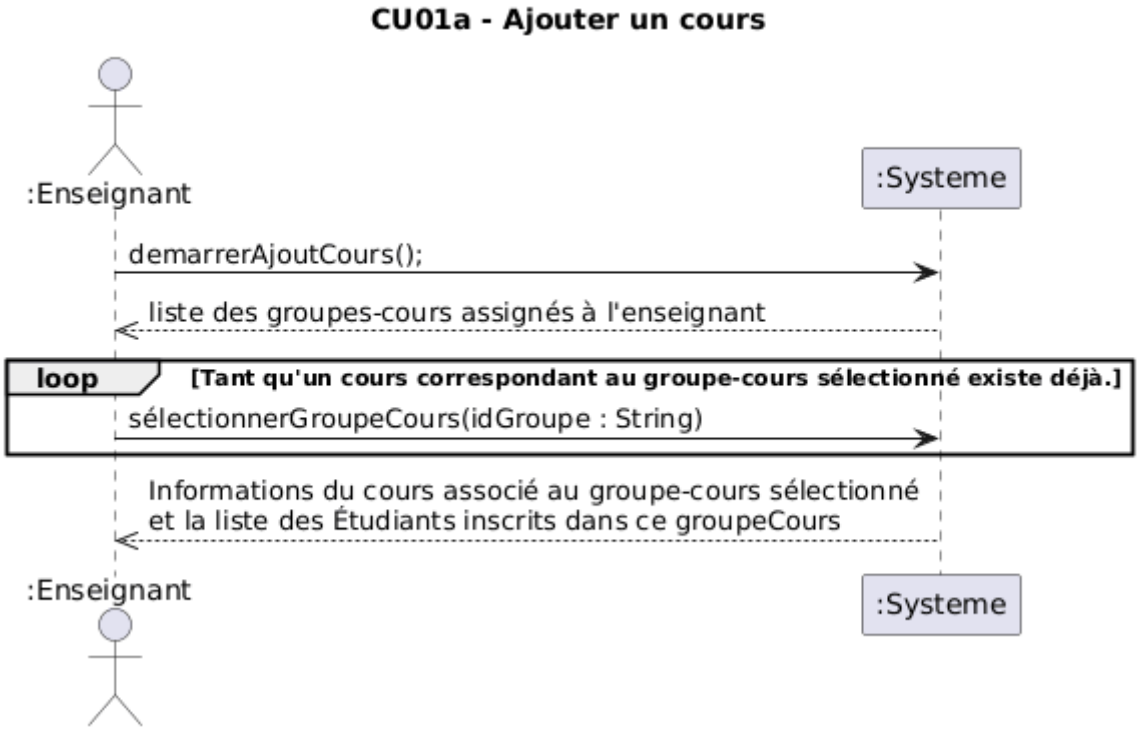
Diagramme de classe TPLANT

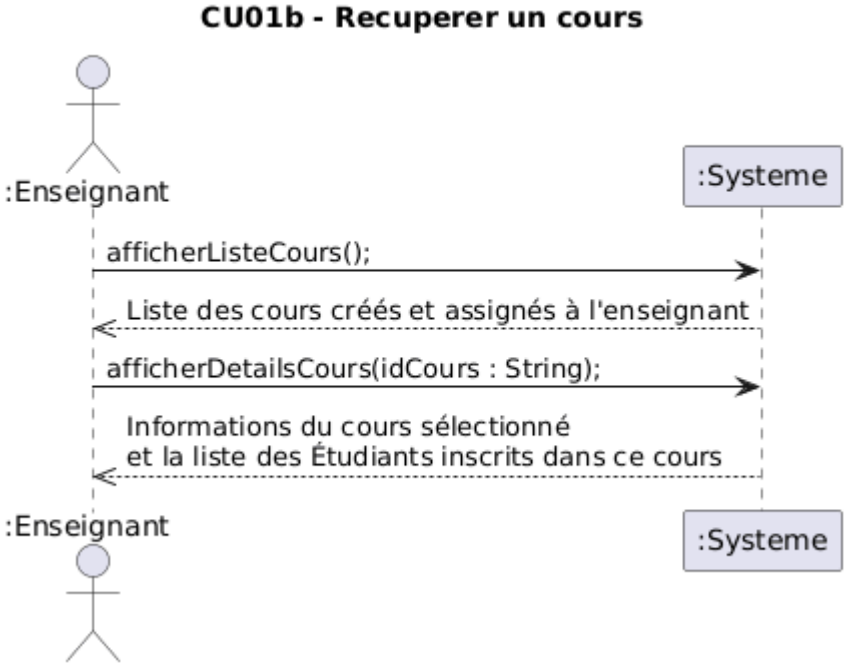
- Générer un diagramme de classe avec l'outil TPLANT et commenter celui-ci par rapport à votre MDD.
- https://www.npmjs.com/package/tplant

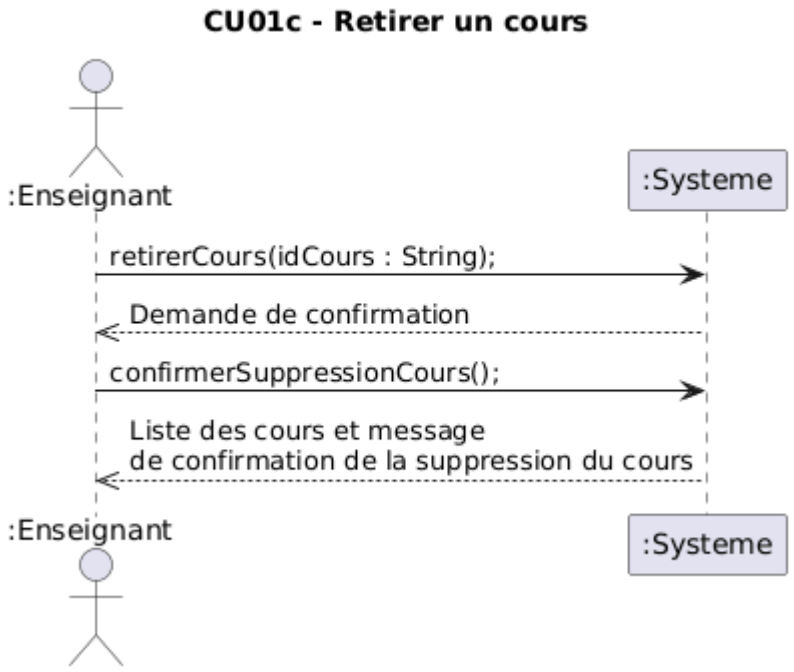
Retour sur la correction du rapport psrécédent

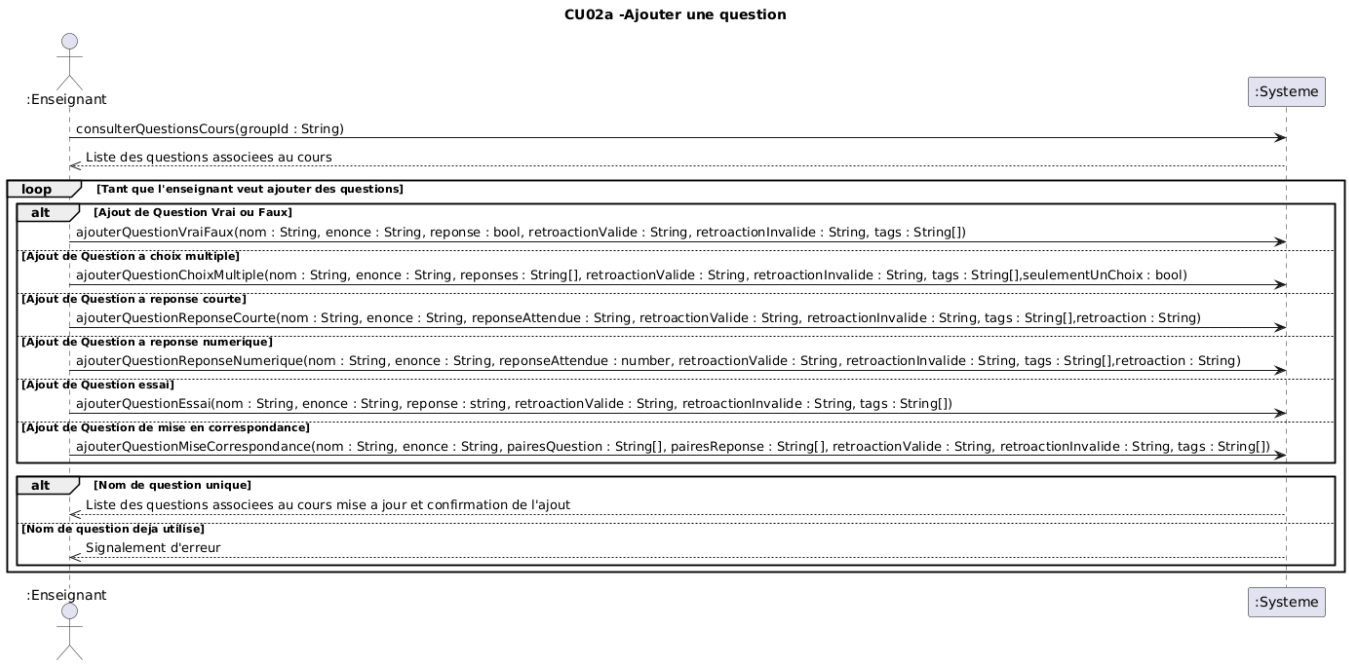
DSS Mis à Jour

Les dss suivant ont été mis a jout afin de corriger les erreurs qui avait été trouvé durant l'itération précédente. (Titres de cu manquants dans le titre et types de questions différents pas pris en compte dans le dss)


CU01a - Ajouter un cours
:Enseignant
:Systeme
demarrerAjoutCours();
liste des groupes-cours assignés à l'enseignant
loop
[Tant qu'un cours correspondant au groupe-cours sélectionné existe déjà.]
sélectionnerGroupeCours(idGroupe : String)
Informations du cours associé au groupe-cours sélectionné
et la liste des Étudiants inscrits dans ce groupeCours
:Enseignant
:Systeme


CU01b - Recuperer un cours
:Enseignant
:Systeme
afficherListeCours();
Liste des cours créés et assignés à l'enseignant
afficherDetailsCours(idCours : String);
Informations du cours sélectionné
et la liste des Étudiants inscrits dans ce cours
:Enseignant
:Systeme





Contrats Mis à Jour

Contrat CO01 - Démarrer Ajout d'un Cours

Les références croisées ont été ajoutés, précisé la forme de la liste

Opération: demarrerAjoutCours()

Références croisées:

- CU01a - Ajouter un cours
- DSS - Ajouter un cours

Préconditions:

- L'Enseignant doit être authentifié.
- Le service SGB est accessible.

PostConditions: HomeController.listeCours contiens la liste des cours associé a l'enseignant authentifié (Array de GroupeCoursSGA)

Contrat CO02 - Sélectionner un Cours

Les références croisées ont été ajoutés,Précisé ce qui est associé au cours (information de l'étudiant en provenance de)

Opération: sélectionnerGroupeCours(idGroupe : String)

Références croisées:

- Contrat CO01 - Démarrer Ajout Cours
- CU01a - Ajouter un cours
- DSS - Ajouter un cours

Préconditions:

- L'Enseignant est authentifié.
- Un jeton d'authentification valide est présent dans la session.
- La liste des groupes-cours assignés à l'Enseignant a été récupérée préalablement via demarrerAjoutCours()

PostConditions:

- Une instance c : Cours a été créée.
- c a été associée à l'Enseignant authentifié.
- Les Informations des étudiants (InfoEtudiant) inscrit à ce groupe-cours étaient associés a c.
- Les informations du groupe-cours(horaire, local, etc.) ont étés enregistrées dans c.

Contrat CO03 - Afficher la liste des cours

Les références croisées ont été ajoutés, précisé l'instance dans la précondition **Opération:** afficherListeCours()

Références croisées:

- CU01b afficher les détails d'un cours
- DSS récuperer cours

Préconditions:

- L'Enseignant est authentifié.
- Un jeton d'authentification valide est présent dans la session

PostConditions:

Contrat CO04 - Afficher les détails d'un cours

Les références croisées ont été ajoutés, précondition au présent

Opération: afficherDetailsCours(idCours: String)

Références croisées:

- CU01b afficher les détails d'un cours
- DSS récuperer cours

Préconditions:

- L'Enseignant a au moins un cours qui lui est assigné.

PostConditions:

Contrat CO05 - Retirer un cours

Les références croisées ont été ajoutés, Le nom de l'operation a été modifié dans le DSS, la postcondition a été supprimé (le cours récupéré est celui qui sera effacé dans l'operation suivante)

Opération: retirerCours(idCours : String)

Références croisées:

- Contrat CO03 - Afficher la liste des cours
- CU01c retirer un cours
- DSS retirer un cours

Préconditions: L'Enseignant est authentifié. L'Enseignant a récupéré un cours (Cu01b)

PostConditions:

Contrat CO06 - Confirmation de la suppression d'un cours

Les références croisées ont été ajoutés

Opération: confirmerSuppressionCours()

Références croisées:

- Contrat CO05 - Retirer un cours
- CU01c retirer un cours
- DSS retirer un cours

Préconditions: L'Enseignant est authentifié. L'Enseignant a récupéré un cours (Cu01b)

PostConditions: Le cours (et seulement ce cours) a été supprimé du système SGA

Contrat CO07 - Gestion de Question

Le contrat est suprimé puisque la fonction n'existe plus

Contrat CO08 - Ajouter une question vrai/faux

Les références croisées ont été ajoutés,les tags ont été ajouté dans la déclaration

---

Opération: ajouterQuestionVraiFaux(nom : String, enonce : String, reponse : bool, retroactionValide : String, retroactionInvalide : String, tags : String[]) : void

Références croisées:
CU02a – Ajouter question
DSS – Ajouter une question
MDD – Question, Cours

Préconditions:

- L'Enseignant.token n'est pas vide.
- Un cours c est sélectionné.

PostConditions:

- Une instance `qvf` de `Question` a été créée.
- `qvf.nom` est devenu `nom`.
- `qvf.énoncé` est devenu `énoncé`.
- `qvf.vérité` est devenu `vérité`.
- `qvf.rétroactionValide` est devenu `rétroactionVrai`.
- `qvf.rétroactionInvalide` est devenu `rétroactionFaux`.
- `qvf` a été associée au `Cours` courant via l'association *contient*.

Contrat CO09 - Ajouter une question d'autre type

Les références croisées ont été ajoutés,les tags ont été ajouté dans la déclaration

---

Opération: ajouterQuestionChoixMultiple(nom : String, enonce : String, reponses : String[], retroactionValide : String, retroactionInvalide : String, tags : String[],seulementUnChoix : bool)

Références croisées: CU02a - Ajouter question DSS - Ajouter une question MDD - Questions, Cours

Préconditions:

- L'Enseignant est authentifié
- Un cours courant est sélectionné.
- Le nom de la question n'existe pas déjà dans la banque de questions du cours courant.

Postconditions:

- Une instance `q` de `Question` a été créée
- `q.nom` est devenu `nom`
- `q.énoncé` est devenu `énoncé`
- `q.type` est devenu `type` (attribut indiquant le type de question)
- `q.rétroactionValide` est devenu `rétroactionValide`
- `q.rétroactionInvalide` est devenu `rétroactionInvalide`

- `q` a été associée au `Cours` courant via l'association *contient*
- Pour chaque élément `t` dans `tags`, une instance de `tags` a été créée ou récupérée et associée à `q` via l'association *catégorisé par*

Vérification finale

- [ ] Vous avez un seul MDD
  - [ ] Vous avez mis un verbe à chaque association
  - [ ] Chaque association a une multiplicité
- [x] Vous avez un DSS par cas d'utilisation
  - [x] Chaque DSS a un titre
  - [x] Chaque opération synchrone a un retour d'opération
  - [x] L'utilisation d'une boucle (LOOP) est justifiée par les exigences
- [ ] Vous avez autant de contrats que d'opérations système (pour les cas d'utilisation nécessitant des contrats)
  - [ ] Les postconditions des contrats sont écrites au passé
- [ ] Vous avez autant de RDCU que d'opérations système
  - [ ] Chaque décision de conception (affectation de responsabilité) est identifiée et surtout **justifiée** (par un GRASP ou autre heuristique)
  - [ ] Votre code source (implémentation) est cohérent avec la RDCU (ce n'est pas juste un diagramme)
- [ ] Vous avez un seul diagramme de classes
- [ ] Vous avez remis la version PDF de ce document dans votre répertoire
- [x] Vous avez regardé cette petite présentation pour l'architecture en couche et avez appliqué ces concepts